

TechNova

Sistema de Gestão Comercial para Loja de Eletrônicos

Entrega 1 — Modelagem e Arquitetura

Disciplina: Desenvolvimento Web

Professor(a): Felipe Pires Ferreira

Equipe: Marcos Henrique, João Cesar

1. Descrição do Sistema

O **TechNova** é um Sistema de Gestão Comercial (SGC) desenvolvido para atender às necessidades operacionais de uma loja de eletrônicos. O sistema abrange o controle de clientes, produtos (smartphones, notebooks, periféricos, acessórios, componentes etc.), registro de vendas, controle de estoque e geração de relatórios gerenciais.

O contexto real simulado é o de uma loja física de eletrônicos de pequeno a médio porte, onde funcionários realizam atendimentos presenciais e precisam consultar estoque, cadastrar clientes, registrar vendas e acompanhar o desempenho comercial por meio de relatórios.

O sistema é composto por uma API REST desenvolvida em **Django REST Framework (DRF)**, uma interface web construída com **Django Templates + Bootstrap**, autenticação via **JWT**, e persistência em banco de dados **SQL Server**.

2. Requisitos Funcionais

ID	Requisito
RF01	O sistema deve permitir o cadastro, edição, listagem e exclusão de clientes
RF02	O sistema deve validar CPF único e e-mail válido no cadastro de clientes
RF03	O sistema deve impedir a exclusão de clientes com vendas registradas
RF04	O sistema deve permitir o cadastro, edição, listagem e exclusão de produtos
RF05	O sistema deve impedir preço negativo e controlar estoque mínimo
RF06	O sistema deve impedir venda quando o estoque for insuficiente
RF07	O sistema deve permitir o registro de vendas com múltiplos itens
RF08	O valor total da venda deve ser calculado automaticamente
RF09	O estoque deve ser atualizado automaticamente após cada venda
RF10	O sistema deve impedir vendas sem itens
RF11	O sistema deve gerar relatórios de vendas por período

ID	Requisito
RF12	O sistema deve gerar relatórios de vendas por cliente
RF13	O sistema deve exibir gráficos anuais de vendas
RF14	O sistema deve permitir login com username e senha
RF15	O sistema deve diferenciar perfis ADMIN e FUNCIONARIO
RF16	O sistema deve permitir recuperação de senha por e-mail

3. Requisitos Não Funcionais

ID	Categoria	Requisito
RNF01	Segurança	Senhas armazenadas com BCrypt
RNF02	Segurança	Autenticação via Token JWT com tempo de expiração
RNF03	Segurança	Rotas protegidas por autenticação e controle de perfil
RNF04	Desempenho	O sistema deve responder às requisições em menos de 2 segundos
RNF05	Manutenibilidade	Código organizado em arquitetura de camadas
RNF06	Portabilidade	Executável em qualquer SO com Python 3.10+ e acesso ao SQL Server
RNF07	Usabilidade	Interface responsiva e intuitiva via Bootstrap
RNF08	Versionamento	Projeto versionado no GitHub com commits organizados
RNF09	Documentação	Documentação técnica completa (diagramas, manual, README)
RNF10	Interoperabilidade	Toda comunicação via API REST retornando JSON

4. Arquitetura Proposta

4.1 Arquitetura em Camadas

O sistema segue uma **Arquitetura em Camadas (Layered Architecture)** com separação clara de responsabilidades:

Camada de Apresentação
Django Templates + Bootstrap 5
Consome a API REST via fetch/axios
Camada de API (Controllers)
Django REST Framework
Endpoints REST, serialização JSON
Camada de Negócio (Services)
Validações, cálculos, regras de negócio
Camada de Persistência (Models)
Django ORM + SQL Server (mssql-django)

4.2 Visão Geral dos Componentes

- **Frontend (Apresentação):** Django Templates renderizam as páginas HTML. JavaScript consome a API REST para operações dinâmicas (CRUD, vendas, relatórios).
- **API REST (Controladores):** DRF expõe endpoints para todas as operações. Retorna JSON. Protegida por JWT.
- **Serviços (Negócio):** Módulos Python com a lógica de negócio (cálculo de totais, validação de estoque, regras de exclusão).
- **Modelos (Persistência):** Django ORM mapeia as entidades para o banco SQL Server via driver `mssql-django`.

4.3 Fluxo de uma Requisição

```
Navegador → Django Template → fetch("/api/clientes/") → DRF ViewSet
  → Service (validação) → Model (ORM) → SQL Server
  → Response JSON → Template atualiza a tela
```

4.4 Tecnologias

Componente	Tecnologia
Linguagem	Python 3.10+
Framework Web	Django 5.x
API REST	Django REST Framework 3.15+
Banco de Dados	SQL Server 2019+
Driver BD	mssql-django + pyodbc
Autenticação	Simple JWT (djangorestframework-simplejwt)
Frontend	Django Templates + Bootstrap 5
Gráficos	Chart.js
Versionamento	Git + GitHub
Testes	pytest + Django TestCase

5. Padrões de Projeto Utilizados

5.1 Repository Pattern (via Django ORM)

O Django ORM atua como uma implementação implícita do padrão Repository, abstraindo o acesso ao banco de dados. Os Models encapsulam as queries, e os ViewSets/Services consomem os dados sem conhecer detalhes do SQL.

Justificativa: Facilita a manutenção e permite trocar o banco de dados sem alterar a lógica de negócio.

5.2 Service Layer

A lógica de negócio é isolada em módulos de serviço (`services.py`), separando-a dos ViewSets (controllers). Exemplo: `VendaService` contém a lógica de cálculo de total e atualização de estoque.

Justificativa: Evita controllers "gordos", melhora a testabilidade e a reutilização de código.

5.3 Serializer Pattern (DRF)

Serializers do DRF funcionam como DTOs (Data Transfer Objects), controlando a entrada e saída de dados da API, realizando validações e transformações.

Justificativa: Garante que os dados trafegados pela API estejam sempre no formato correto e validados.

5.4 Template Method (Autenticação JWT)

O fluxo de autenticação segue um template definido: receber credenciais → validar → gerar token → retornar. A biblioteca Simple JWT implementa esse padrão com pontos de extensão.

Justificativa: Permite customizar a geração de tokens sem reescrever o fluxo completo.

5.5 Strategy Pattern (Relatórios)

Os relatórios utilizam diferentes estratégias de filtragem (por período, por cliente, por produto) aplicadas sobre a mesma base de dados de vendas.

Justificativa: Permite adicionar novos tipos de relatório sem modificar os existentes.

6. Endpoints da API REST

Método	Endpoint	Descrição	Acesso
POST	/api/auth/login	Autenticação, retorna token JWT	Público
POST	/api/auth/refresh	Renovação do token	Autenticado
GET	/api/clientes	Listar todos os clientes	Autenticado
POST	/api/clientes	Cadastrar novo cliente	Autenticado
PUT	/api/clientes/{id}	Atualizar cliente	Autenticado
DELETE	/api/clientes/{id}	Excluir cliente	ADMIN
GET	/api/produtos	Listar todos os produtos	Autenticado
POST	/api/produtos	Cadastrar novo produto	Autenticado
PUT	/api/produtos/{id}	Atualizar produto	Autenticado
DELETE	/api/produtos/{id}	Excluir produto	ADMIN
GET	/api/vendas	Listar vendas	Autenticado
GET	/api/vendas/{id}	Detalhar venda	Autenticado
POST	/api/vendas	Registrar nova venda	Autenticado
GET	/api/relatorios	Relatório por período/cliente	Autenticado

7. Casos de Uso

UC01 — Realizar Login

Ator: Usuário (Admin ou Funcionário)

Pré-condição: Possuir cadastro no sistema

Fluxo principal: Informar username e senha → sistema valida credenciais → retorna token JWT → redireciona ao painel

Fluxo alternativo: Credenciais inválidas → exibe mensagem de erro

UC02 — Gerenciar Clientes

Ator: Funcionário ou Admin

Fluxo principal: Cadastrar, editar, listar ou excluir cliente

Regras: CPF deve ser único; e-mail deve ser válido; exclusão bloqueada se houver vendas vinculadas

UC03 — Gerenciar Produtos

Ator: Funcionário ou Admin

Fluxo principal: Cadastrar, editar, listar ou excluir produto

Regras: Preço não pode ser negativo; estoque mínimo deve ser controlado e alertado

UC04 — Registrar Venda

Ator: Funcionário ou Admin

Pré-condição: Ao menos um cliente e um produto cadastrados

Fluxo principal: Selecionar cliente → adicionar itens com quantidade → confirmar venda

Regras: Estoque verificado antes da confirmação; valor total calculado automaticamente; estoque atualizado após venda; venda sem itens é impedida

UC05 — Gerar Relatórios

Ator: Admin

Fluxo principal: Selecionar filtros (período, cliente) → visualizar dados tabulares e gráficos

Regras: Gráficos anuais de vendas com Chart.js; filtros por período e por cliente

UC06 — Recuperar Senha

Ator: Usuário

Fluxo principal: Informar e-mail cadastrado → receber link de redefinição → criar nova senha

Regras: Link com token temporário; senha criptografada com BCrypt

8. Estrutura do Projeto

```
technova/
├── manage.py
├── requirements.txt
├── README.md
├── .gitignore
├──
├── docs/
│   ├── entrega1_modelagem.pdf
│   ├── diagrama_classes.png
│   ├── diagrama_banco.png
│   └── script_banco.sql
├──
├── technova/                                (Configuração Django)
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├──
├── apps/
│   ├── usuarios/                          (Autenticação e perfis)
│   ├── clientes/                          (CRUD de clientes)
│   ├── produtos/                         (CRUD de produtos e estoque)
│   └── vendas/                           (Vendas e relatórios)
├──
├── templates/                             (Templates HTML + Bootstrap)
│   ├── base.html
│   ├── login.html
│   ├── clientes/
│   ├── produtos/
│   ├── vendas/
│   └── relatorios/
├──
└── static/                                (CSS, JS, Imagens)
    ├── css/
    ├── js/
    └── img/
```

9. Repositório GitHub

Link: <https://github.com/MarcosVogado/technova>

O repositório está organizado com commits descritivos seguindo o padrão semântico (docs: , feat: , fix:), README atualizado e documentação na pasta /docs .